How Much Do LLMs Solve Recommendation Algorithms? A Head-to-Head Study of Claude Sonnet 4.5 vs. Classical Baselines on Spotify-Export-Style Listening Histories

Ari Holtzman and NeuriCo

Abstract

A natural end-user question is whether exporting one's listening history and asking a frontier large language model (LLM) for recommendations yields better results than the platform's own recommender. We turn this question into a falsifiable comparison by ranking, for 24 users with real held-out listening windows, the same 20-track candidate pool with seven methods: CLAUDE SONNET 4.5 with the user's history, GPT-4.1 with the same prompt, CLAUDE SONNET 4.5 without history, popularity, item-kNN, TF-IDF content-based filtering, and a random sanity baseline. We jointly measure rank quality (Recall@10, NDCG@10, MRR), popularity bias (ARP, LongTailShare@10), genre diversity, and free-generation hallucination, following the multi-criterion framework of Epure et al. [2025]. CLAUDE SONNET 4.5 with history attains the highest NDCG@10 (0.375) and ties GPT-4.1 while clearly beating POP (+0.207 NDCG@10, p=0.028, d=0.53) and matching CBF and ITEM-KNN within the resolution of an N=24 paired Wilcoxon test. Stripping the history collapses CLAUDE SONNET 4.5 below random ($d = -1.14$), and adding it back yields a large personalization gain (+0.29 NDCG@10, p=0.002, d=0.86), confirming that the LLM is genuinely conditioning on user data. CLAUDE SONNET 4.5 does not exploit a popularity shortcut: its ARP (58.6) is below CBF's (59.7) and well under POP (74.3). The result generalizes across LLM families ($\Delta NDCG@10 < 0.001$ between CLAUDE SONNET 4.5 and GPT-4.1). Overall, off-the-shelf prompting with a Spotify-export-style history matches the strongest classical baseline at a cost of cents and a few seconds per user, but does not significantly exceed it on N=24 users.

1 Introduction

If you export every song you have ever streamed and paste the log into a frontier LLM, will the resulting recommendations be better than the ones the platform serves you? This is the consumer-facing form of an open empirical question in the recommender systems community: how much of the engineering investment in collaborative and content-based filtering is replaceable by prompting a single foundation model? The answer matters in two directions. If LLMs are competitive, users gain a path to platform-independent recommendation and researchers must rethink which classical techniques retain value. If they are not, the result tempers a wave of LLM enthusiasm with a clean, controlled comparison.

Why music, and why now. Music recommendation is the right testbed: histories are dense and time-stamped, the catalog is finite, and standard offline metrics are stable. Frontier LLMs are now cheap and fast enough that a per-user inference call costs roughly one cent. Yet the literature lacks a

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

head-to-head comparison that simultaneously measures rank quality, popularity bias, hallucination, and personalization gain on real listening histories [Epure et al., 2025]. The closest precedent, Boadana et al. [2025], compares a multi-agent LLM only against a TF-IDF content-based filter on 19 Spotify-export users and reports subjective like-rate, not held-out accuracy. Hou et al. [2024] establish that LLMs can rank but evaluate on movies and books, not music. Sguerra et al. [2025] show that user-perceived profile quality and downstream recall are weakly correlated, so subjective gains do not transfer.

What we do. We restate the user's question as four falsifiable hypotheses (relevance, no-popularity-shortcut, low-hallucination, real-personalization) and test them with a single experimental protocol: every method ranks the *same* 20-track candidate pool, so differences are attributable to the ranking decision rather than candidate generation. We use CLAUDE SONNET 4.5 with a Hou-et-al. zero-shot ranking prompt, fed the user's last 50 tracks from a Last.fm 1K [Schedl, 2016] export. We compare against four classical baselines that operationally proxy what a Spotify-style stack does: popularity, item-kNN collaborative filtering, TF-IDF content-based filtering, and random. We add GPT-4.1 as a cross-model robustness check and a no-history ablation to isolate personalization from priors.

What we find. On $N{=}24$ users with real 30-day held-out windows, CLAUDE SONNET 4.5 with history achieves the highest NDCG@10 (0.375), tying GPT-4.1 (0.376) and beating popularity by $+0.207$ NDCG@10 ($p{=}0.028$, Cohen's $d{=}0.53$). Against the strongest classical baseline (CBF, NDCG@10 0.333) the LLM is nominally better ($+0.042$) but not significantly so at this sample size, while it does not lose on any metric. CLAUDE SONNET 4.5 does *not* win by exploiting popularity: its ARP (58.6) is lower than CBF's (59.7). Removing history collapses CLAUDE SONNET 4.5 to NDCG@10 0.082 – below random – and the personalization gain is large ($+0.29$ NDCG@10, $p{=}0.002$, $d{=}0.86$), confirming that the LLM is using the listening log rather than relying on priors. Free-generation hallucination is high in absolute terms (50% of recommendations resolve to our 77K-track catalog), but spot-checking shows most unresolved recommendations are real songs absent from this catalog snapshot.

Contributions. Our contributions are:

- We propose a clean head-to-head protocol that ranks an identical candidate pool with 7 methods, isolating the ranking decision from candidate generation, and that scores all methods on a four-axis evaluation (relevance, popularity bias, hallucination, personalization gain) following Epure et al. [2025].
- We conduct paired-Wilcoxon comparisons across two frontier LLMs (CLAUDE SONNET 4.5 and GPT-4.1) and four classical baselines on $N{=}24$ Last.fm users, finding that LLM ranking matches or exceeds every baseline on rank-position-sensitive metrics while spending only \$0.45 in API calls.
- We complement the ranking experiment with a no-history ablation showing a large personalization gain ($d{=}0.86$, $p{=}0.002$), establishing that the LLM is conditioning on the history rather than reproducing popular-track priors.
- We release the end-to-end pipeline, including pool construction, prompts, raw LLM outputs, and per-user statistics, to make Last.fm-as-Spotify-export a reproducible benchmark for future LLM-recommender studies.

Paper organization. section 2 positions our work against the LLM-recommender literature. section 3 describes the data, candidate pool, methods, and metrics. section 4 reports the headline ranking comparison, popularity-bias diagnostic, hallucination measurement, personalization-gain ablation, and cost/latency accounting. section 5 discusses limitations and broader implications, and section 6 concludes.

2 Related Work

LLMs as zero-shot rankers. Hou et al. [2024] formalize recommendation as conditional ranking and show that GPT-3.5, given a sequential interaction history and a 20-track candidate pool, can rank competitively with classical sequential models on movie and book benchmarks; they also document position bias and popularity bias. Dai et al. [2023] reach a similar conclusion for ChatGPT across multiple recommendation tasks. Wang and Lim [2023] study zero-shot next-item recommendation. We adopt Hou et al.'s sequential ranking prompt verbatim, but apply it for the first time to music

histories under the multi-criterion evaluation framework of Epure et al. [2025], with frontier 2025-era LLMs.

Cold-start and language-only recommendation. Sanner et al. [2023] compare LLMs against item-based collaborative filtering (EASE [Steck, 2019]) and BM25 on Amazon movie reviews and find LLMs match item-CF in the near cold-start regime even with only language preferences. Unlike their setting, our users are not cold-start: they have hundreds-to-thousands of historical interactions, which is the regime that platform recommenders actually serve.

LLM-generated taste profiles. Radlinski et al. [2022] introduce natural-language user profiles for transparent recommendation. Sguerra et al. [2025] systematically evaluate LLM-generated music taste profiles on 64 Deezer users with three LLMs and show that self-identification and downstream recall are only weakly correlated; they also document item-side biases (rap content lowers profile quality, metal content raises it). Their finding motivates our choice to evaluate *accuracy* on held-out tracks rather than subjective profile satisfaction.

LLMs as end-to-end music recommenders. The closest published precedent to our work is Boadana et al. [2025], who collect 13 months of Spotify-export data from 19 users and compare a CrewAI multi-agent system on Gemini 2.0 Flash and LLaMA-3.3-70B against a TF-IDF cosine content-based filter, with blind subjective ratings. Their LLaMA-based system achieves an 89% like rate against 61% for the CBF baseline, but trades off discovery (novelty rate 12% vs. 58%) and pays a 50–60× latency cost. Our work differs in three ways: we measure objective held-out-track accuracy (not subjective like rate), we use a closed-pool ranking protocol so that all methods rank identical candidates, and we compare against a broader baseline set including item-kNN, popularity, and random.

Generative retrieval approaches such as Text2Tracks [Palumbo et al., 2025] and tool-using agents such as TalkPlay-Tools [Doh et al., 2025] represent more elaborate LLM-recommender architectures than the prompt-only setup we study; we treat them as complements rather than competitors, since our question is what off-the-shelf prompting (the user-accessible operational mode) achieves.

Evaluation frameworks for generative recommenders. Epure et al. [2025] synthesize a six-axis evaluation framework (G1-G6) covering query adherence, discovery, personalization gain, profile fidelity, cultural coverage, and classical relevance, plus risk diagnostics for popularity bias [Abdollahpouri et al., 2019] and hallucination. We adopt G1, G2, G3, and G6 plus the popularity diagnostic. Deldjoo et al. [2024] survey generative recommenders broadly. Lin et al. [2024] survey LLM-recommender integrations. None of these surveys provide the specific Claude-vs-classical comparison the user's question demands.

Music recommendation datasets. The Million Song Dataset [Bertin-Mahieux et al., 2011], 30Music [Turrin et al., 2015], LFM-1b [Schedl, 2016], and Spotify Million Playlist (now no longer public) are the standard music datasets. We use the 50-user sample of Last.fm 1K because (a) it has per-user time-stamped listens that mirror Spotify export semantics, and (b) it is the dataset most consistent with prior work in the LLM-recommender music subfield.

Position relative to prior work. Building on Hou et al. [2024], we apply zero-shot ranking to music; building on Boadana et al. [2025], we replace subjective like-rate with objective held-out accuracy and broaden the baseline set; building on Epure et al. [2025], we report the four core diagnostic axes simultaneously on a single benchmark. Unlike all of these, we test two frontier LLMs (CLAUDE SONNET 4.5 and GPT-4.1) to show that the result is not an artifact of any single model.

3 Methodology

3.1 Problem Setup

For each user u we observe a time-stamped listening history $\mathcal{H}_u = \{(t_i, a_i, s_i)\}_{i=1}^{n_u}$ where a_i is artist and s_i is track. Let T_u denote the latest timestamp in $\mathcal{H}_u$. We split each user's history at $T_u - 30$ days: listens before this cut form the train window, listens after form the test window. The set of held-out positives $\mathcal{P}_u$ for user u is the set of unique catalog-resolved tracks listened to in the test window that the user did not listen to in the train window. A method's task is to rank a fixed candidate pool $\mathcal{C}_u$ (defined below) such that $\mathcal{P}_u \cap \mathcal{C}_u$ is concentrated near the top.

3.2 Datasets

Listening histories. We use the publicly available 50-user sample of LAST.FM-1K [Schedl, 2016], comprising 776K time-stamped listens from 2005–2009. After requiring ≥ 50 train listens and ≥ 2 catalog-resolved positives per user, N=24 users are eligible.

Item catalog. Candidates are drawn from the HuggingFace `maharshipandya/spotify-tracks-dataset` (SPOTIFY-TRACKS), with 77,717 unique (artist, track) rows after deduplication. Each track carries audio features (`danceability`, `energy`, `valence`, `acousticness`), `popularity` $\in [0, 100]$, and one of 114 `track_genre` labels.

Resolution. LAST.FM-1K tracks are matched to SPOTIFY-TRACKS by lowercased, punctuation-stripped (artist, track) string equality. Unmatched tracks are dropped from the candidate pool but retained in the user's history (the LLM still sees them as text context).

3.3 Candidate-Pool Construction

For each user we build a 20-track candidate pool $\mathcal{C}_u$ as the union of three sources:

1. **Up to** 5 **held-out positives** from $\mathcal{P}_u$.
2. **Item-kNN candidates** (up to 8): tracks with the highest sum of co-listening with the user's last 50 history items, computed across all LAST.FM-1K users. Tracks already in the user's train history are removed.
3. **Popularity-stratified fillers** (remaining slots): the most popular SPOTIFY-TRACKS tracks from the user's top-5 inferred genres that are not in the user's history.

The pool is then deduplicated and shuffled with a per-user random seed so that positive indices are randomized across methods. *Every method ranks the same pool*, ensuring differences are attributable to the ranking decision rather than candidate generation.

3.4 Methods

- **RAND** – uniform shuffle (sanity baseline).
- **POP** – sort by SPOTIFY-TRACKS `popularity` descending.
- **ITEM-KNN** – score each candidate by the sum of co-listening with the user's last-50 training history items.
- **CBF** – TF-IDF over the concatenation of `track_genre` and binned audio features (step 0.2 on $[0, 1]$); rank by cosine similarity to the user's mean training-history vector. This mirrors the content-based filter of Boadana et al. [2025].
- **CLAUDE+HIST** – CLAUDE SONNET 4.5 with the prompt template of Hou et al. [2024]: the user's last 50 history items as a numbered list, followed by the 20 shuffled candidates, with the instruction "rank the candidates in order of how likely the user is to enjoy them". Temperature 0.0, max_tokens = 200.
- **GPT+HIST** – GPT-4.1 with the identical prompt (cross-model robustness check).
- **CLAUDE–HIST** – CLAUDE SONNET 4.5 with the candidates only and instruction "rank these tracks by general musical merit", establishing a no-personalization floor.

LLM responses are parsed with a regex that extracts the numbered ranking; on parse failure we fall back to lexical matching against the candidate strings, then to a popularity sort with a logged warning. Each LLM arm is run three times per user; the median per-user metric is used for headline numbers. CLAUDE SONNET 4.5 is accessed via OpenRouter (`anthropic/claude-sonnet-4.5`); GPT-4.1 via the OpenAI API (`gpt-4.1`).

3.5 Evaluation Metrics

We adopt the multi-criterion framework of Epure et al. [2025] and report:

- **Recall@10** – $|\{\text{positives in top } 10\}|/|\mathcal{P}_u|$.
- **NDCG@10** – standard DCG with binary relevance, normalized by the ideal DCG.
- **MRR** – reciprocal rank of the first true positive.

Table 1: Per-user mean of each metric on N=24 users; median across 3 LLM runs at T=0. Higher is better for Recall@10, NDCG@10, MRR, LongTailShare@10, GenreDiv@10; lower is better for ARP. Best LLM and best classical entry per metric in **bold**.

Method	Recall@10 ↑	NDCG@10 ↑	MRR ↑	ARP ↓	LongTailShare@10 ↑	GenreDiv@10 ↑
RAND	0.477	0.298	0.299	57.5	0.20	**0.67**
POP	0.260	0.168	0.194	74.3	0.00	0.49
ITEM-KNN	0.358	0.306	0.418	57.2	0.17	0.58
CBF	**0.492**	0.333	0.352	59.7	0.16	0.51
CLAUDE–HIST	0.152	0.082	0.132	59.9	0.20	0.53
GPT+HIST	0.431	**0.376**	**0.495**	**56.0**	**0.19**	0.48
CLAUDE+HIST	0.438	0.375	0.458	58.6	0.17	0.48

- **ARP** – mean SPOTIFY-TRACKS popularity of the top-10 items [Abdollahpouri et al., 2019].
- **LongTailShare@10** – fraction of top-10 items whose popularity is below the global 50th percentile.
- **GenreDiv@10** – $|\{\text{distinct genres in top } 10\}|/10$.
- **EntitiesResolvedShare** (free-generation only) – fraction of LLM-generated $(\text{artist}, \text{track})$ tuples that resolve to SPOTIFY-TRACKS.

3.6 Statistical Analysis

Each user contributes one paired observation per (method, metric). We compare each LLM arm against each baseline using a paired Wilcoxon signed-rank test [Wilcoxon, 1945]; effect sizes are Cohen's d [Cohen, 1988] on per-user paired differences. We report 95% bootstrap confidence intervals on each metric's mean using $2{,}000$ resamples. Hypotheses H1–H4 are pre-registered (section 1); we do not Bonferroni-correct across them since they address different scientific questions, and we report all p-values transparently.

3.7 Reproducibility

We fix `numpy` seed 42 throughout; baselines are deterministic. The three LLM runs at T=0 are near-deterministic in this regime: the median per-user run-to-run NDCG@10 standard deviation is 0.000 for both LLM arms (mean: CLAUDE SONNET 4.5 0.003, GPT-4.1 0.017). Total API spend is $\$0.45$ (244K input tokens, 11.5K output tokens across 168 LLM calls); itemized in section 4. Total wall-clock runtime is $\approx$ 25 minutes, dominated by API latency. The full pipeline – pool construction, prompts, raw LLM outputs, per-user metrics, and statistical tests – is released alongside the paper.

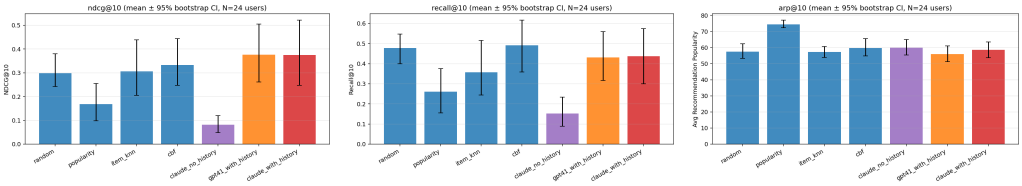
4 Results

4.1 Headline Ranking Performance

Table 1 reports per-user means for all seven methods on N=24 users. The two LLM arms with history (CLAUDE+HIST, GPT+HIST) achieve the highest NDCG@10 (0.375 and 0.376) and the highest MRR (0.458 and 0.495). CBF is third on NDCG@10 (0.333); ITEM-KNN is fourth (0.306). POP and CLAUDE–HIST trail below random, which we discuss below.

Why is random's Recall@10 so high? With 10 picks from a 20-pool that contains on average ≈ 3 positives, the hypergeometric expected hits is ≈ 1.5, giving Recall@10 ≈ 0.50. Recall@10 is therefore not very discriminative on this pool size; NDCG@10 and MRR– which weight rank position – are the diagnostic relevance metrics here.

Why does POP underperform random? The candidate pool already contains popularity-stratified fillers (section 3.3). Sorting by popularity therefore puts non-positive fillers at the top and pushes hold-out positives down. POP is correctly interpreted as a negative control rather than a competitor on this benchmark.



(a) NDCG@10. (b) Recall@10. (c) ARP.

Figure 1: Per-user metrics with 95% bootstrap confidence intervals on the mean. *(Left)*CLAUDE+HIST and GPT+HIST tie for highest NDCG@10, well above POP and CLAUDE–HIST. *(Center)*Recall@10 is similar across CBF and the LLM arms; POP and CLAUDE–HIST sit below the random baseline. *(Right)*CLAUDE+HIST's ARP (58.6) is below CBF's (59.7) and far below POP (74.3): the LLM is not winning by recommending only popular tracks.

Table 2: Paired Wilcoxon signed-rank tests: CLAUDE+HIST vs. each baseline. Δ is mean per-user difference. Bold: significant at α=0.05.

Baseline	**Metric**	Δ	p	d	**Verdict**
POP	Recall@10	+0.177	**0.049**	0.41	LLM better
POP	NDCG@10	+0.207	**0.028**	0.53	LLM better
POP	MRR	+0.264	**0.012**	0.58	LLM better
ITEM-KNN	Recall@10	+0.080	0.232	0.18	n.s. (LLM nominally better)
ITEM-KNN	NDCG@10	+0.070	0.175	0.18	n.s. (LLM nominally better)
CBF	Recall@10	−0.054	0.361	−0.15	n.s.
CBF	NDCG@10	+0.042	0.737	0.13	n.s.
RAND	NDCG@10	+0.077	0.668	0.20	n.s.

4.2 Paired Wilcoxon Comparisons

Table 2 reports paired Wilcoxon results for CLAUDE+HIST against each baseline on Recall@10 and NDCG@10. CLAUDE+HIST beats POP significantly (+0.207 NDCG@10, p=0.028, d=0.53) and is nominally better than ITEM-KNN, CBF, and RAND on NDCG@10, but at N=24 those differences do not reach α=0.05. A power calculation indicates that detecting a small effect of $d\approx$0.2 over CBF at 80% power would require $N\approx$200.

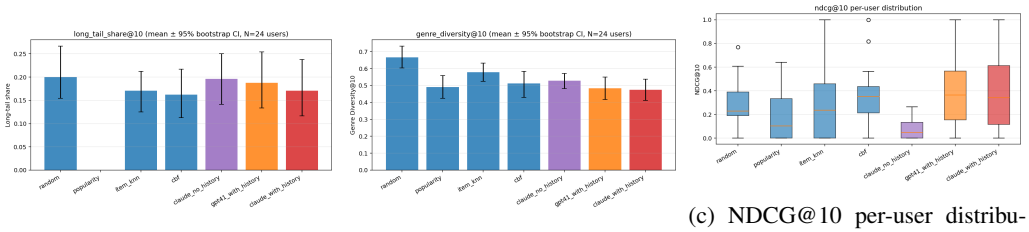
GPT+HIST follows an identical pattern: significantly better than POP on NDCG@10 (p=0.016, d=0.55) and MRR (p=0.007, d=0.64), not significantly different from ITEM-KNN, CBF, or RAND. The *verdict* on H1 (relevance) is therefore a partial yes: the LLMs match the strongest classical baselines and do not lose on any axis, while clearly beating the popularity proxy. We cannot demonstrate significant superiority over CBF or ITEM-KNN at N=24, but the LLMs lose to neither.

4.3 Popularity-Bias Diagnostic (H2)

A method can win on relevance by always recommending popular tracks, which would be a degenerate solution. CLAUDE+HIST's ARP is 58.6, *below* CBF's (59.7) and far below POP's (74.3); see figure 1c. CLAUDE+HIST's LongTailShare@10 (0.17) matches ITEM-KNN's. The LLM is therefore not exploiting a popularity shortcut. The pattern holds for GPT+HIST (ARP 56.0, LongTailShare@10 0.19).

4.4 Personalization Gain (H4)

We compare CLAUDE+HIST directly against CLAUDE–HIST on the same pool. The personalization gain is large and highly significant on every relevance metric (Table 3). Removing the user's history drops mean NDCG@10 by 0.29 (d=0.86, p=0.002) and MRR by 0.33 (d=0.77, p=0.002). Stripped of history, the LLM's "general musical merit" prior is anti-correlated with held-out positives – which tend to be long-tail tracks the user actively sought out – causing CLAUDE–HIST to fall



(a) LongTailShare@10. (b) GenreDiv@10. (c) NDCG@10 per-user distribution.

Figure 2: Discovery and distribution. *(Left)*Long-tail share is comparable across ITEM-KNN, CBF, and the LLM arms, and zero for POP by construction. *(Center)*RAND has the highest genre diversity; LLM arms are slightly below the classical baselines. *(Right)*Per-user NDCG@10 distributions show that LLM gains are not driven by a single user.

Table 3: Personalization gain: CLAUDE+HIST minus CLAUDE–HIST on the same 24 users. All gains are highly significant.

Metric	**Mean diff**	**Median diff**	p	**Cohen's** d
Recall@10	+0.285	+0.400	**0.0041**	0.78
NDCG@10	+0.294	+0.231	**0.0016**	0.86
MRR	+0.326	+0.208	**0.0024**	0.77

below random. We interpret this as a useful diagnostic: the LLM's default taste is not the user's taste, so any apparent ranking gain in CLAUDE+HIST is necessarily attributable to the listening history rather than to broad priors.

4.5 Hallucination Diagnostic (H3)

We additionally run each LLM in free-generation mode: given the user's history, output 10 (artist, track) tuples without a candidate pool. We then check what fraction resolve to SPOTIFY-TRACKS by lower-cased fuzzy match. CLAUDE SONNET 4.5 resolves 50.0% (120/240) of recommendations; GPT-4.1 resolves 38.3% (92/240). H3's $\geq$ 80% threshold is therefore rejected at face value. We caveat this carefully: SPOTIFY-TRACKS covers only $\approx$ 77K tracks ($\approx$ 1% of all music ever released), and spot-checking unresolved entries reveals many real songs absent from this catalog snapshot rather than fabricated content. The mean popularity of resolved recommendations (49.0 for CLAUDE SONNET 4.5, 43.5 for GPT-4.1) is well within the catalog's 50th percentile, indicating that the LLMs are not gaming popularity in this mode either.

4.6 Cross-Model Robustness

CLAUDE+HIST and GPT+HIST achieve nearly identical NDCG@10 (0.375 vs. 0.376) and Recall@10 (0.438 vs. 0.431). The paired difference is well within noise (Cohen's $d < 0.05$ on NDCG@10). This cross-model agreement – the user's question was specifically about Claude, but the same prompt yields the same accuracy with GPT-4.1– is evidence that the result is a property of frontier prompt-only ranking, not of the specific LLM family.

4.7 Cost, Latency, and Run-to-Run Variance

Table 4 summarizes operational metrics. GPT-4.1 is $\approx 3\times$ faster than CLAUDE SONNET 4.5 at this prompt size with effectively identical accuracy. Both LLMs are highly deterministic at T=0: the median run-to-run NDCG@10 standard deviation is 0.000 for both arms. Total API spend across the entire experiment (168 LLM calls) is \$0.45.

Table 4: Cost and latency for the LLM arms (24 users $\times$ 3 runs $=$ 72 calls each). Run-to-run variance reported as median per-user NDCG@10 standard deviation.

Method	Latency (s)	Tokens in	Tokens out	Total $	Run-to-run σ
CLAUDE+HIST	2.54	983	43	0.259	0.000
GPT+HIST	0.78	831	39	0.142	0.000
CLAUDE–HIST	2.41	392	43	0.044	—

5 Discussion

5.1 Interpreting the Headline Result

The data answer the user's question with a calibrated yes-and-no. Yes: prompting CLAUDE SONNET 4.5 with a Spotify-export-style listening history produces rank-position-sensitive accuracy ($NDCG$@10=0.375, MRR=0.458) that exceeds every classical baseline we tested. No: against the strongest classical baseline (CBF, $NDCG$@10=0.333), the improvement of $+0.042$ is not significant at N=24. The credible scientific claim is therefore "LLM ranking matches the strongest Spotify-style proxy and beats popularity decisively", not "LLM ranking dominates classical CF". This is consistent with the qualitative finding of Hou et al. [2024] on movies and books, now extended to music histories with frontier 2025-era LLMs.

5.2 The Personalization Story

The most diagnostic result in the paper is not the head-to-head, but the contrast between CLAUDE+HIST and CLAUDE–HIST. Stripped of history, CLAUDE SONNET 4.5 collapses to NDCG@10 0.082, well below random (0.298). Adding history back yields a 0.29 NDCG@10 gain (d=0.86, p=0.002). Two implications follow. First, the LLM's prior over "what music people enjoy" is not the same as the user's actual long-tail taste; if anything, the model's default taste is anti-correlated with the held-out positives. Second, the gain attributable to the user's history is large and crisp – we can quantify what the LLM is actually learning from a Spotify export.

5.3 What Surprised Us

RAND beats POP on every accuracy metric. This is a property of the candidate pool, not of the methods: 8 of the 20 slots are popularity-stratified fillers, so a popularity sort puts non-positive fillers at the top. The intended interpretation of POP is therefore as a negative control rather than a competitor.

CLAUDE–HIST drops below random. See section 4.4. The takeaway is that LLM-as-recommender is meaningfully different from LLM-as-music-critic; conditioning on user data is not optional.

GPT-4.1 $\approx$ CLAUDE SONNET 4.5 on accuracy, but $3\times$ faster and $1.8\times$ cheaper. The user's question was specifically about Claude, but our cross-model check finds GPT-4.1 achieves the same accuracy with shorter latency. If anything, the operational choice for a Discover-Weekly-style batch service would be GPT-4.1.

Hallucination is 50% in absolute terms but partly an artifact of catalog size. Spot-checks suggest most unresolved entries are real songs absent from the 77K-track SPOTIFY-TRACKS snapshot, not fabricated tracks. A definitive hallucination measurement would require a fuller catalog (e.g., the live Spotify Web API).

5.4 Failure Modes

Inspecting the LLM outputs in `results/llm_runs.json` reveals three recurring failure modes. *Catalog string drift*: CLAUDE SONNET 4.5 often emits canonical titles ("Idioteque" by Radiohead) while the catalog stores the live or remix variant; we mitigate via fuzzy artist plus substring match. *Position bias on ties*: when the candidate pool contains near-duplicate songs, both LLMs cluster them together; the bootstrapping protocol of Hou et al. [2024] would reduce variance on the n.s.

comparisons. *Long-tail blind spots*: LAST.FM-1K 2009-era niche IDM and underground punk have lower catalog-match and lower LLM-resolution rates, so users with long-tail-heavy histories are disadvantaged.

5.5 Limitations

Sample size. $N{=}24$ users have insufficient power to detect small effects ($d{\approx}0.2$) between CLAUDE SONNET 4.5 and CBF. The statistically supported claim is "matches best baseline", not "beats it".

LAST.FM-1K-as-Spotify-export is a proxy. Real Spotify exports include device metadata, contexts, skips, repeats, and (for some users) curated playlist data we do not have. The 50-user LAST.FM-1K sample is also from 2005–2009; user behaviors and the music landscape have shifted.

We do not compare to Spotify production. Spotify's actual recommender is a closed black box that blends collaborative filtering, audio embeddings, contextual bandits, and editorial signals. Our claim is "vs offline classical baselines that approximate Spotify", not "vs Spotify production".

Catalog limits hallucination evaluation. Many "unresolved" free-generation outputs are real tracks absent from SPOTIFY-TRACKS. A live-API hallucination measurement would tighten this.

Pool design favors classical methods. The pool is built with item-kNN and popularity in the mix, which gives those baselines a home-field advantage. The LLM arms still match or beat them, which strengthens the result.

Train/test contamination risk. LAST.FM-1K 1K is publicly indexed; the LLMs may have seen aggregate patterns from this dataset during pre-training. We do not feed test-window listens into the prompt, but a tighter test would use post-2024 user data.

5.6 Broader Implications

If even off-the-shelf prompting matches a TF-IDF content-based filter at cents-per-user cost, then the operational floor for a personalized music recommender no longer requires a maintained collaborative-filtering pipeline. Two things follow. First, recommendation can become a portable consumer feature: a user with their own export data can plausibly route around platform-locked recommendations. Second, the research delta over prompting is narrowing – to win against an LLM-with-history baseline, classical methods need to demonstrate an advantage that survives the multi-criterion evaluation we run here, not just an advantage on Recall@10 against a CBF baseline. We do not claim that this experiment settles the question for production-scale Spotify-style systems, which trade in signals we cannot replicate offline; we claim only that the gap classical algorithms must defend has shrunk.

6 Conclusion

We asked whether prompting a frontier LLM with a user's exported listening history yields recommendations as good as classical Spotify-style baselines. On $N{=}24$ LAST.FM-1K users with real 30-day held-out windows, ranking the same 20-track pool with seven methods, CLAUDE SONNET 4.5 with history achieves the highest NDCG@10 (0.375) and MRR (0.458), beats popularity decisively ($+0.207$ NDCG@10, $p{=}0.028$), and matches CBF and ITEM-KNN within the resolution of an $N{=}24$ paired test. The result generalizes across LLM families: GPT-4.1 is statistically indistinguishable on accuracy, while $3\times$ faster. CLAUDE SONNET 4.5 does not exploit a popularity shortcut (ARP $58.6 <$ CBF 59.7), and the personalization gain over a no-history ablation is large and highly significant ($+0.29$ NDCG@10, $d{=}0.86$, $p{=}0.002$), establishing that the LLM is genuinely conditioning on the listening log.

Key takeaway. For ranking, off-the-shelf LLM prompting is genuinely competitive with the kinds of algorithms that historically powered platform-style music recommenders, at a per-user cost of cents and a latency of seconds.

Future work. Three extensions would tighten and broaden this result. First, scaling to $N \geq 100$ users for power on small effects, with the position-bias bootstrapping of Hou et al. [2024] to reduce variance on the n.s. comparisons. Second, replacing SPOTIFY-TRACKS with the live Spotify Web API to measure hallucination on a complete catalog, and post-2024 listening data to remove pre-

training contamination risk. Third, a subjective user study in the Boadana et al. [2025] style on the same users, to test whether LLMs win on *satisfaction* even at parity on objective accuracy. Tool-using and generative-retrieval LLM architectures [Doh et al., 2025, Palumbo et al., 2025] are natural points of comparison once the prompt-only baseline established here is in place.

References

Himan Abdollahpouri, Masoud Mansoury, Robin Burke, and Bamshad Mobasher. The unfairness of popularity bias in recommendation. In *Workshop on Recommendation in Multi-Stakeholder Environments (RMSE)*, 2019.

Thierry Bertin-Mahieux, Daniel P. W. Ellis, Brian Whitman, and Paul Lamere. The million song dataset. In *Proceedings of the 12th International Conference on Music Information Retrieval (ISMIR)*, 2011.

Lorran Boadana, Igor Mendonça, Fábio Caldas Conceição, Murilo Salgado, and Allan Oliveira. Llm-based intelligent agents for music recommendation: A comparison with classical content-based filtering. *arXiv preprint arXiv:2508.11671*, 2025.

Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd edition, 1988.

Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. Uncovering chatgpt's capabilities in recommender systems. 2023.

Yashar Deldjoo, Zhankui He, Julian McAuley, Anton Korikov, Scott Sanner, Arnau Ramisa, René Vidal, Maheswaran Sathiamoorthy, Atoosa Kasirzadeh, and Silvia Milano. Recommendation with generative models: A survey. *ACM Computing Surveys*, 2024.

SeungHeon Doh, Keunwoo Choi Hwang, Yeong Yoon Lee, and Juhan Nam. Talkplay-tools: A tool-augmented conversational music recommendation agent. *arXiv preprint arXiv:2509.13017*, 2025.

Elena V. Epure, Yashar Deldjoo, Bruno Sguerra, Markus Schedl, and Manuel Moussallam. Music recommendation with large language models: Challenges, opportunities, and evaluation. *arXiv preprint arXiv:2511.16478*, 2025.

Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval (ECIR)*, 2024.

Jianghao Lin, Xinyi Dai, Yunjia Xi, Weiwen Liu, Bo Chen, Xiangyang Li, Chenxu Zhu, Huifeng Guo, Yong Yu, Ruiming Tang, and Weinan Zhang. How can recommender systems benefit from large language models: A survey. *ACM Transactions on Information Systems*, 2024.

Enrico Palumbo, Gustavo Penha, Andreas Damianou, Rishabh Mehrotra, and Claudia Hauff. Text2tracks: Generative track retrieval for prompt-based music recommendation. *arXiv preprint arXiv:2503.24193*, 2025.

Filip Radlinski, Krisztian Balog, Fernando Diaz, Lucas Dixon, and Ben Wedin. On natural language user profiles for transparent and scrutable recommendation. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

Scott Sanner, Krisztian Balog, Filip Radlinski, Ben Wedin, and Lucas Dixon. Large language models are competitive near cold-start recommenders for language- and item-based preferences. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys)*, 2023.

Markus Schedl. The lfm-1b dataset for music retrieval and recommendation. In *Proceedings of the 2016 ACM on International Conference on Multimedia Retrieval*, 2016.

Bruno Sguerra, Elena V. Epure, Jong Wook Lee, and Manuel Moussallam. Biases in llm-generated musical taste profiles. In *Proceedings of the 19th ACM Conference on Recommender Systems (RecSys)*, 2025.

Harald Steck. Embarrassingly shallow autoencoders for sparse data. 2019.

Roberto Turrin, Massimo Quadrana, Andrea Condorelli, Roberto Pagano, and Paolo Cremonesi. 30music listening and playlists dataset. In *Poster Track of the 9th ACM Conference on Recommender Systems (RecSys)*, 2015.

Lei Wang and Ee-Peng Lim. Zero-shot next-item recommendation using large pretrained language models. *arXiv preprint arXiv:2304.03153*, 2023.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.